REGISTRO DE DECISÃO DE ARQUITETURA

ADR-001: Modernização e Evolução da Plataforma de Gestão de Fluxo de Caixa

Data: 10 de outubro de 2026

Status: Proposto

Autoria: Alessandra Santos

Domínio de Negócio: Gestão Financeira (Cash Flow Management)

SEÇÃO 1: CONTEXTO - O CENÁRIO ATUAL (AS-IS)

1.1 Descrição do Problema e Cenário Legado

O sistema atual (***AS-IS***) de Gestão de Fluxo de Caixa foi concebido como uma arquitetura monolítica fortemente acoplada, hospedada em um único servidor *on-premise*. O comerciante acessa a interface web diretamente, com autenticação básica, para realizar duas operações vitais: o registro de lançamentos (débitos/créditos) e a consulta do relatório de consolidado diário.

As limitações deste modelo tornaram-se impeditivos para o crescimento do negócio e violam o requisito não funcional primário: a resiliência.

Dores de Negócio e Limitações Técnicas Atuais:

- **Acoplamento Síncrono Total:** Como o sistema é único, se a rotina de consolidação (leitura) falhar ou consumir muitos recursos, a operação de frente de caixa (escrita) é interrompida, paralisando o comerciante.
- **Gargalos de Banco de Dados:** Existe apenas uma persistência única (*RDBMS SQL Server*), escritas e leituras concorrentes pesadas geram *locks* (bloqueio de tabelas) e *deadlocks* (travamento cruzado), resultando em *timeouts* durante os horários de pico.
- **Risco de Perda de Dados:** Não há mecanismo de *buffer* (filas). Falhas no servidor ou banco resultam na perda da transação no momento da tentativa de gravação.
- **Gestão de Falhas e Deploy da aplicação:** Logs são armazenados em disco local (em formato de texto), sem telemetria centralizada. Qualquer alteração exige o *re-deploy* de todo o monolito (o que sinaliza um alto risco para o sistema de gestão de fluxo de caixa).

1.2 Mapeamento de Domínios Funcionais (AS-IS)

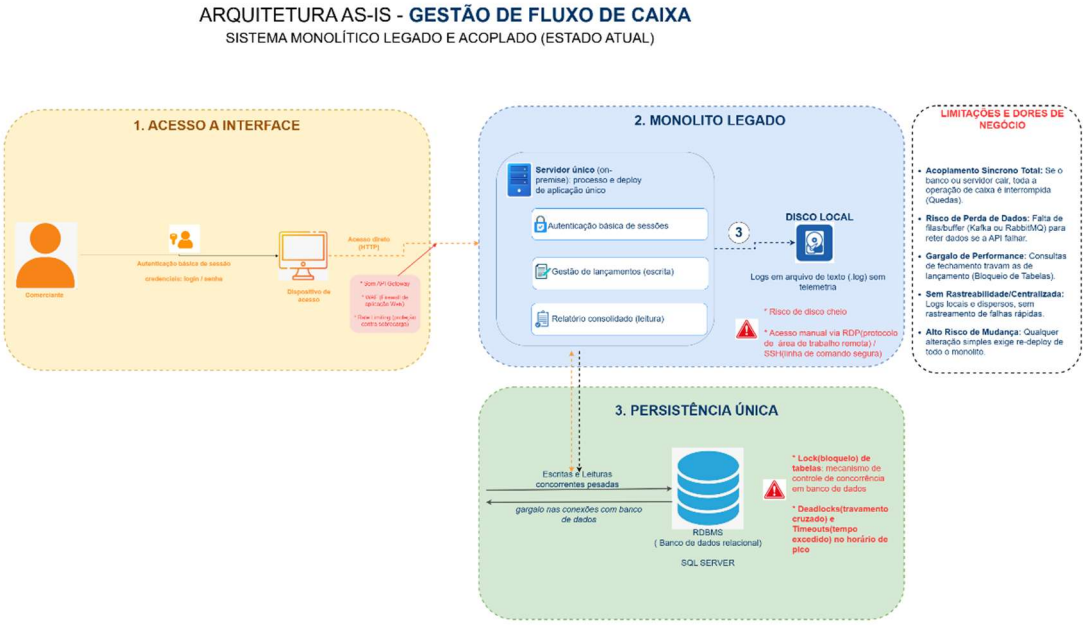


Figura 1 – Arquitetura AS-IS – Gestão de Fluxo de Caixa

A figura 1 ilustra a arquitetura de estado atual (AS-IS) para a gestão de fluxo de caixa, fundamentado num sistema monolítico legado e acoplado, que opera a partir de um único servidor *on-premise*. O fluxo tem início com o comerciante realizando um acesso direto via HTTP através do seu dispositivo, utilizando uma autenticação básica de sessão por *login* e senha, sem a proteção de componentes de borda ou de segurança como API Gateway, WAF ou limitação de taxa (*Rate Limiting*). Após o acesso, todas as funções de negócio, como a escrita para gestão de lançamentos e a leitura para relatórios consolidados, são processadas nesse mesmo monolito e competem por uma base de dados relacional (RDBMS) única em SQL Server, enquanto os *logs* de telemetria são apenas guardados localmente num disco em ficheiros de texto, exigindo um acesso manual via RDP(Remote Desktop Protocol – tradução: protocolo de ambiente de trabalho remoto) ou SSH(Secure Shell – tradução: terminal seguro).

Esta estrutura monolítica acarreta severas limitações para o negócio, destacando-se o acoplamento síncrono total, que implica que qualquer quebra no servidor ou na base de dados resulte na interrupção imediata de toda a operação de caixa. A persistência centralizada origina graves gargalos de performance, uma vez que as operações concorrentes pesadas de leitura e escrita provocam bloqueios de tabelas, travamentos cruzados (*deadlocks*) e esgotamento do tempo de resposta (*timeouts*) durante os períodos de pico. Adicionalmente, a arquitetura padece de um elevado risco de perda de dados devido à ausência de sistemas intermédios de broker de mensageria (exemplos: Kafka, ServiceBus ou RabbitMQ) para reter informação em caso de falha da API, possui falta de rastreabilidade devido aos *logs* dispersos e impõe um elevado risco de mudança, pois qualquer pequena alteração exige o *re-deploy* integral da aplicação.

A tabela a seguir representa a divisão do domínio principal em subdomínios e capacidades:

Domínio Principal	Subdomínios (Acoplados)	Capacidades de Negócio	Limitação Atual
Gestão Financeira	Acesso e Segurança	Autenticação Básica de Sessão	Inseguro, sem proteção contra sobrecarga.
	Lançamentos (Escrita)	Registrar Débitos/Créditos	Compete por recursos com as leituras.
	Consolidado (Leitura)	Visualizar Saldo Diário	Consultas complexas travam o banco.

Para proporcionar uma compreensão clara e estruturada da arquitetura atual, utilizamos o Modelo C4, que atua como diferentes níveis de visões sobre o sistema, clarificando o entendimento por outras perspectivas:

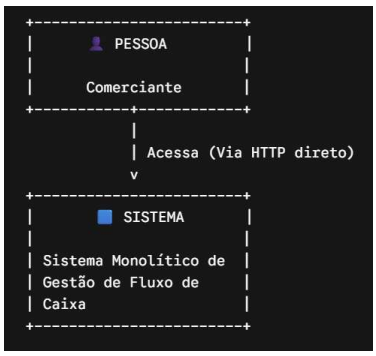


Figura 2 – Nível 1 – Diagrama de Contexto (visão C4 Model)

A figura 2 apresenta a visão macro, ilustrando a interação direta do comerciante com a plataforma.

A figura a seguir representa a visão de diagrama de container:

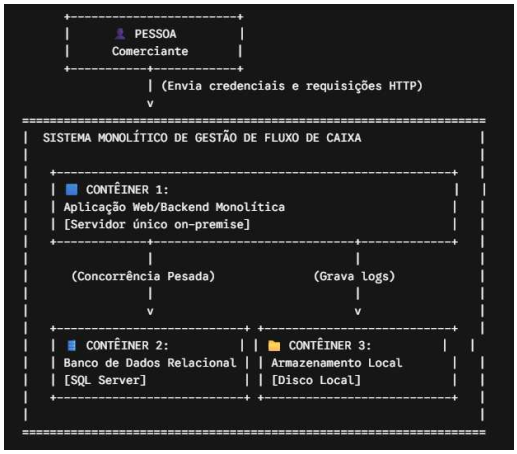


Figura 3 – Nível 2 – Diagrama de Container (visão C4 Model)

A figura 3 representa o nível 2 (Contentores), revelamos a arquitetura de implantação, expondo a dependência de um servidor monolítico único e de uma base de dados centralizada.

Por fim, a figura a seguir apresenta a visão de diagramas de componentes:

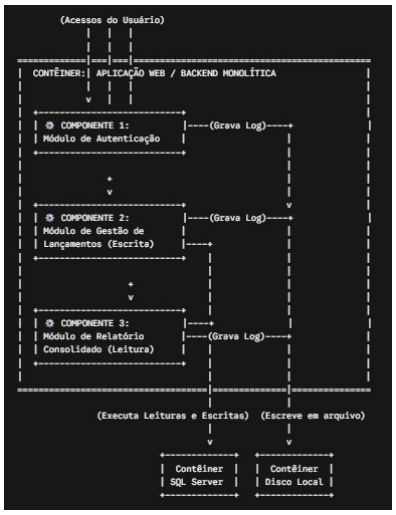


Figura 4 – Nível 3 – Diagrama de componentes (visão C4 Model)

A figura 4 representa o nível 3 (Componentes) e detalha o interior da aplicação, mapeando os módulos acoplados de autenticação, lançamentos e relatórios, o que evidencia a origem técnica das atuais dores do negócio.

SEÇÃO 2: DECISÃO

Após análise da arquitetura atual, ficou entendível que tentar modernizar o sistema inteiro de uma só vez traria riscos altíssimos e demoraria muito tempo para resolver os problemas diários do comerciante.

De forma estratégica, apresenta-se duas etapas de divisão e evolução do sistema atual.

A **Visão Tática** funciona como uma transição rápida, com foco em praticidade de tecnologias e codificação, cujo objetivo central é atacar as dores mais urgentes, desacoplando os relatórios dos lançamentos para evitar que o sistema continue a travar. O grande benefício desta abordagem incremental é mitigar os riscos operacionais, garantindo a estabilidade da plataforma e entregando valor imediato ao negócio. Outro fator importante nessa visão é a abordagem sobre a ingestão dos dados do legado para uma estrutura de banco de dados moderna(utilizando o conceito de CDC(captura de dados alterados), escalável e pronta para futuras migrações para cloud-native*(*computação em nuvem).

A **Visão Target** justifica-se pela necessidade estratégica de atingir a maturidade máxima em resiliência, segurança e escalabilidade, libertando a organização do fardo operacional de gerir infraestrutura. Ao migrar integralmente para um ecossistema *cloud-native* ancorado em serviços geridos (*PaaS:* plataforma como serviço e *Serverless*: sem servidor), a arquitetura deixa de ser apenas um suporte transacional e passa a atuar como um habilitador de inovação, permitindo a integração futura de inteligência artificial e conciliação bancária automatizada. O grande benefício desta visão target a é a eliminação do esforço de manutenção técnica, garantindo elasticidade automática para lidar perfeitamente com os picos de tráfego exigidos pelo negócio e segurança de nível corporativo por padrão.

SEÇÃO 2.1: VISÃO TÁTICA - DESACOPLAMENTO E MENSAGERIA

Para mitigar os desafios imediatos de performance do negócio e preparar o ambiente para a escalabilidade na nuvem, a Arquitetura Tática implementa o padrão CQRS (*Command Query Responsibility Segregation* – tradução: Segregação de Responsabilidade de Comandos e Consultas) a nível lógico e de infraestrutura, em conjunto com a arquitetura *Event-Driven(*arquitetura orientada a eventos).

A arquitetura foi dividida em domínios isolados, permitindo escalabilidade independente:

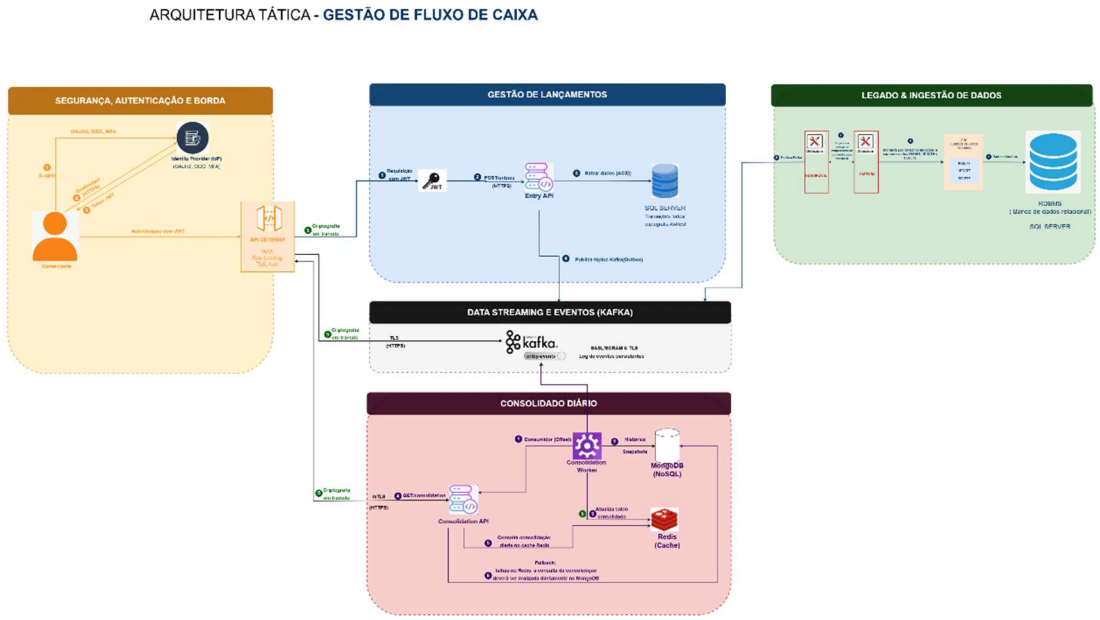


Figura 5 – Arquitetura tática

A figura 5 apresenta a Visão Tática (MVP) da nossa arquitetura de Gestão de Fluxo de Caixa, o fluxo que descreve a entrada de um lançamento até à sua ingestão e consolidação assenta num modelo distribuído orientado a eventos (Event-Driven) e tira partido do padrão CQRS aliado à estratégia de migração CDC (Change Data Capture).

O processo ocorre de forma assíncrona para garantir a resiliência (meta rigorosa de máximo 5% de perda) e o desempenho (capacidade para 50 RPS), isolando completamente o "caixa do lojista" do "relatório do gerente".

No modelo C4, o entendimento da Visão Tática apresenta-se nos seguintes níveis:

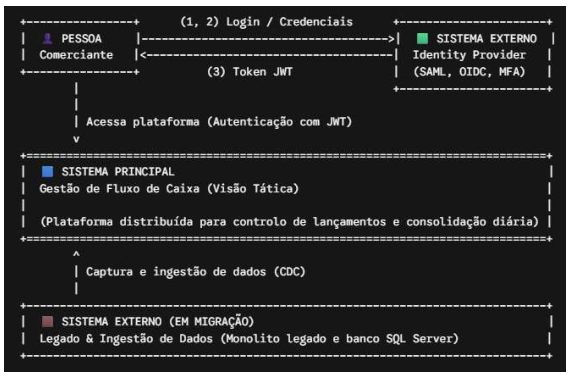


Figura 6 – Diagrama de contexto

A figura 6 apresenta o nível 1(diagrama de contexto – visão macro) de forma que a plataforma de Gestão de Fluxo de Caixa (Visão Tática) ao centro, interagindo com o

comerciante e sistemas externos (o provedor de identidade e o sistema legado em fase de transição).

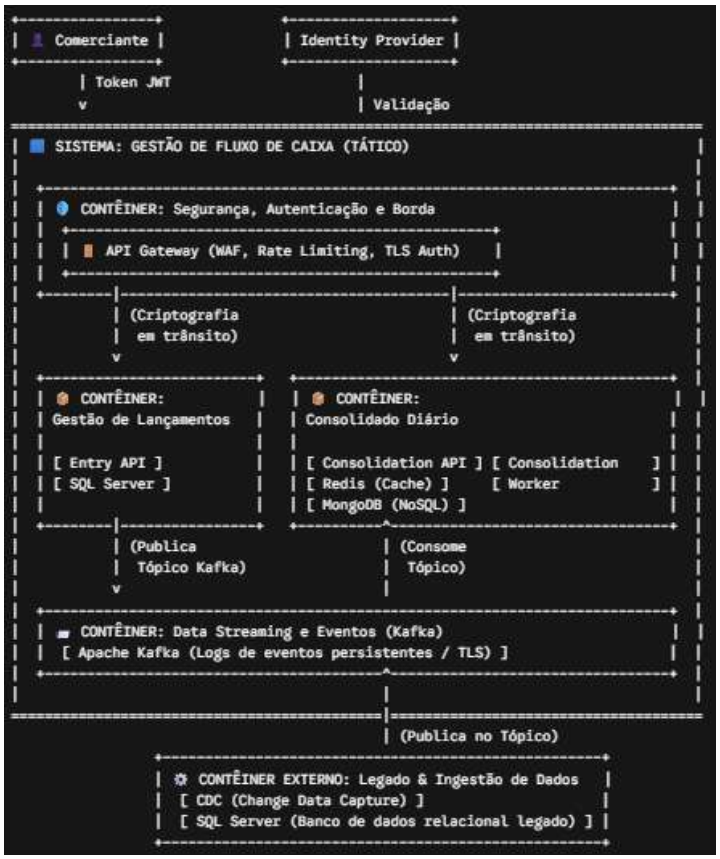


Figura 7 – Diagrama de container

A figura 7 retrata o nível 2 do sistema principal (caixa azul do Nível 1) para revelar as grandes unidades de execução, representadas no seu desenho pelos diferentes domínios e tecnologias (APIs, Brokers, Bancos de Dados).

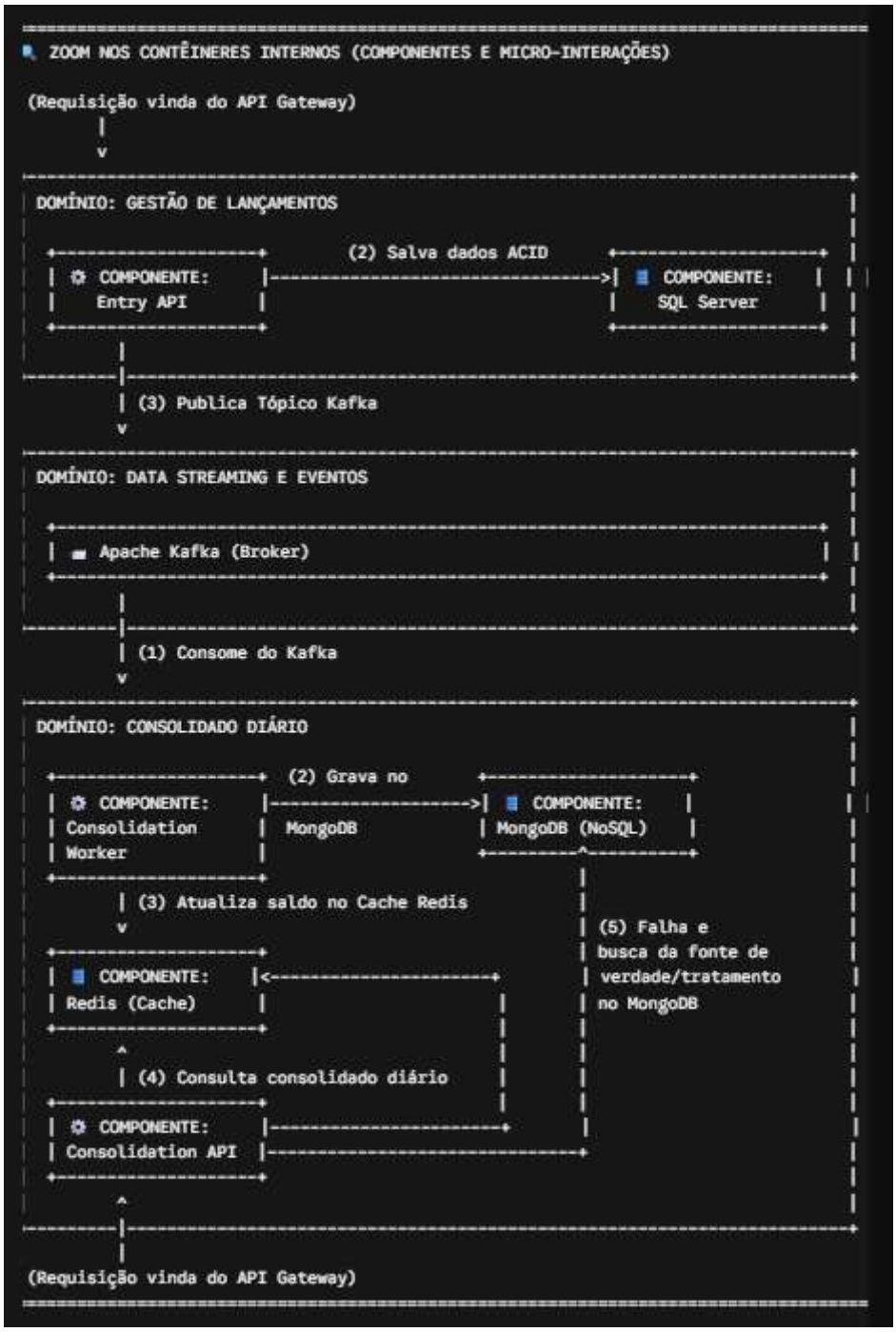


Figura 8 – Diagrama de componentes

A figura 8 apresenta o nível 3 (diagrama de componentes), com os detalhes e interações internas exatas dos fluxos de Gestão de Lançamentos e Consolidação Diário, mostrando como as APIs se conectam aos bancos de dados de leitura e escrita e ao barramento de eventos.

Para facilitar a compreensão e implementação dessa visão, segue descritivo do fluxo:

1. Borda e Autenticação (A Porta de Entrada)

1. **Acesso e Segurança:** O comerciante acede à aplicação para registar um lançamento (Débito ou Crédito). O pedido HTTPS é interceptado no nível de "Segurança, Autenticação e Borda".

2. **Validação de Identidade (Zero-Trust):** O comerciante autentica-se num Identity Provider, que emite um **Token JWT**.

3. **Filtragem no API Gateway:** A requisição validada pelo Token JWT chega ao API Gateway (protegido por WAF e encriptação TLS 1.3). O Gateway aplica as políticas de *Rate Limiting* e encaminha o tráfego da operação de escrita de forma segura para o domínio de *Command*.

2. Gestão de Lançamentos (O Lado Command / Escrita)

4. **Processamento pela Entry API:** A requisição chega à **Entry API** (um serviço totalmente autónomo e dedicado à inserção de dados). A API valida a regra de negócio do lançamento.

5. **Gravação Transacional (ACID):** A *Entry API* salva o registo no banco de dados relacional **SQL Server** (ou PostgreSQL). Esta etapa garante a conformidade ACID: o dado financeiro está seguro, o dinheiro não se perde. A partir deste exato milissegundo, a API devolve um código "200 OK" ou "201 Created", libertando o processo para o próximo cliente.

3. Ingestão e Backbone de Eventos (Estratégia CDC e Mensageria)

6. **Captura de Mudança de Dados (CDC):** Na base de ingestão de dados SQL SERVER, é configurado o fluxo via Debezium/Transaction Log), para cada *insert, delete* ou *update* realizado é capturado por um processo de CDC de forma não-intrusiva.

7. **Padrão *Transactional Outbox***: O CDC transforma esta alteração física no banco de dados num evento de negócio puro e publica este evento no *broker* de mensageria, o **Apache Kafka**.

8. **Retenção e Resiliência (Kafka):** O Kafka armazena as mensagens num tópico com forte persistência em disco. Esse tópico será consumido pelo Consolidation Worker, cuja responsabilidade é salvar e ou atualizar esses dados no banco MongoDB(para histórico) e no Redis(para consultas rápidas).

4. Consolidado Diário (O Lado Query / Leitura)

9. **Consumo Assíncrono:** O **Consolidation Worker** atua no domínio de relatórios, subscreve o tópico do Kafka. Ele recolhe a mensagem de forma assíncrona assim que for publicado.

10. **Materialização do Saldo:** O *Worker* pega o valor monetário contido em uma mensagem publicada no tópico, calcula o novo saldo final para aquele dia/comerciante e grava os dados frios (*snapshots* históricos) numa base de dados NoSQL (**MongoDB**).

11. **Atualização em Tempo Real no Cache:** Simula e atualiza imediatamente o mesmo saldo no cache em memória, o **Redis**.

12. **Consulta Rápida pela API:** Quando o comerciante pede para ver o Relatório de Consolidado, o API Gateway encaminha o pedido para a **Consolidation API**. Esta API vai ler diretamente ao **Redis** (onde o tempo de resposta é constante),

devolvendo o saldo atualizado de forma ultrarrápida, cumprindo o limite de 50 RPS sem tocar nem bloquear as tabelas de inserção (SQL Server).

A partir do fluxo de execução da arquitetura na Visão Tática, os requisitos funcionais, não-funcionais e técnicos do sistema são:

Requisitos funcionais:

- **RF-01 (Registro de Movimentação)**: O sistema deve fornecer uma API (Entry API) capaz de receber, validar e registar lançamentos financeiros (débitos e créditos) iniciados pelo lojista.
- **RF-02 (Ingestão Legada - CDC):** O sistema deve capturar de forma assíncrona, através de *Change Data Capture* (CDC), as alterações realizadas na base de dados legada e transformá-las em eventos de negócio.
- **RF-03 (Consolidação de Saldo):** O sistema deve possuir um processo em *background* (*Worker*) que consuma os eventos de lançamentos e processe matematicamente a consolidação do saldo financeiro diário por estabelecimento.
- **RF-04 (Consulta de Relatório):** O sistema deve fornecer uma API (Consolidation API) para que o lojista consulte o saldo consolidado atualizado, de forma independente do fluxo de escrita.

Requisitos não-funcionais:

- **RNF-01 (Desempenho e Throughput):** As APIs devem suportar picos de tráfego de até 50 Requisições Por Segundo (RPS) sem degradação de tempo de resposta. As requisições de consulta de saldo devem ter complexidade temporal O(1) (Tempo constante).
- **RNF-02 (Resiliência e Tolerância a Falhas):** A perda máxima tolerável de eventos/transações é fixada no rigoroso limite de 5%. O sistema de mensageria deve atuar como *buffer* (retenção em disco/fila) para garantir que picos de carga não resultem em perda de dados caso as APIs de leitura estejam indisponíveis.
- **RNF-03 (Isolamento de Domínio / CQRS):** A disponibilidade do domínio de escrita (Gestão de Lançamentos) não pode ser impactada pela indisponibilidade ou lentidão do domínio de leitura (Consolidado Diário), eliminando o "Acoplamento Síncrono Total" do legado.

Requisitos técnicos:

- **RT-01 (Stack de Desenvolvimento):** O código fonte das APIs e do Worker deve ser escrito utilizando a linguagem de domínio do time de engenharia(programação).
- **RT-02 (Bancos de Dados Poliglotas):** O armazenamento deve respeitar a especialidade de cada domínio:
    - *Command:* SQL Server (garantia transacional ACID).
    - *Query (Hot Data):* Redis Cache (para consulta ultrarrápida do saldo).
    - *Query (Cold Data):* MongoDB (NoSQL) para armazenamento de histórico de *snapshots*.

- **RT-03 (Mensageria MVP):** O transporte assíncrono de eventos e o padrão *Transactional Outbox* devem ser implementados sobre o **Apache Kafka** (devido à sua alta retenção em disco comparado ao RabbitMQ).
- **RT-04 (Conteinerização):** A execução e orquestração local (ambiente de desenvolvimento) de toda a infraestrutura e código deverá ser feita exclusivamente via Docker/Docker Compose.

Na proposta de mensurar os custos mensais da presente visão de arquitetura, segue a estimativa mensal:

- **Entry API / Consolidation API / Worker (Compute):** Hospedagem em servidores virtuais (IaaS). Custo: média de R$ 750,00 – R$ 1.500,00/mês.
- **SQL Server:** Licenciamento Microsoft (Pode variar de $0 na versão Express a milhares de dólares no Enterprise). *Custo existente herdado do AS-IS*.
- **Apache Kafka / Zookeeper:** *Open-source (Gratuito)*. Custo atrelado à infraestrutura (Virtual Machine- tradução: máquina virtual). Estimativa IaaS: média 1.250,00/mês.
- **MongoDB:** *Open-source (Community Edition)*. Custo IaaS: média 500,00/mês.
- **Redis:** *Open-source*. Custo IaaS: média 250,00/mês.
- **Total Estimado de Infra (Excluindo licenças SQL):** média de R$ 2.750,00 a R$ 3.500,00/mês.

SEÇÃO 2.2: VISÃO TARGET - ARQUITETURA CLOUD-NATIVE (AZURE)

A Visão Target representa a maturidade arquitetural máxima. O sistema é integralmente migrado para o ecossistema **Microsoft Azure**, utilizando serviços gerenciados (PaaS e Serverless) que reduzem a sobrecarga operacional, aumentam drasticamente a segurança e habilitam inovações via IA.

Importante destacar que a escolha pelo provedor em nuvem Microsoft Azure foi no sentido de demonstrar a facilidade em tornar a aplicação pronta para ser escalável, reutilizável e modernizada, partindo do princípio que a Visão Target busca atender o que há de mais robusto no uso de tecnologias de última geração. Lembrando que o provedor é uma decisão que é tomada pela gestão da empresa e, dependendo da decisão, a Visão Target está pronta para ser adequada por outras tecnologias.

A arquitetura foi dividida em domínios isolados, permitindo escalabilidade independente:

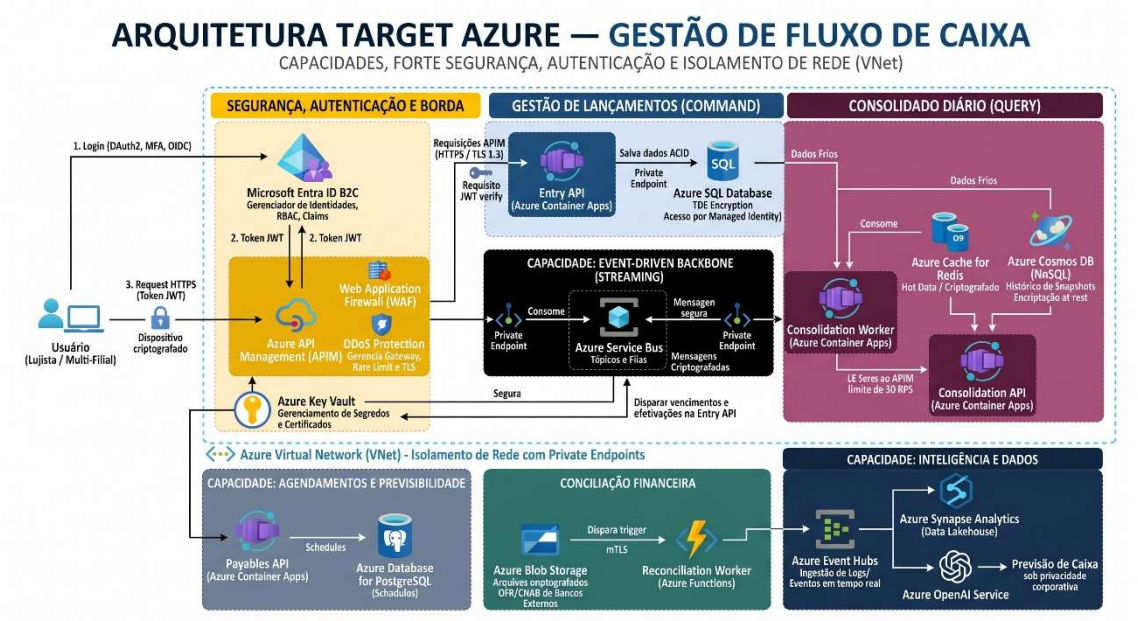


Figura 9– Arquitetura Visão Target Azure – Gestão de Fluxo de Caixa

A figura 9 apresenta o fluxo desta arquitetura divide o sistema em duas vias completamente independentes dentro de uma rede blindada (VNet). Quando o lojista regista um movimento no caixa, a *Entry API* salva a transação de forma definitiva num banco relacional (Azure SQL) e envia um alerta para uma fila de mensagens (Azure Service Bus). Em segundo plano, um robô (*Worker*) lê esse alerta e atualiza o saldo financeiro numa memória ultrarrápida (Redis), que será consumida pela *Consolidation API* na hora de gerar os relatórios. Este desenho é vital para o negócio porque garante que o registo de vendas nunca trave ou saia do ar, mesmo que os sistemas paralelos de relatórios, inteligência artificial ou conciliação bancária estejam sobrecarregados.

Além disso, embora a figura 9 ilustre serviços específicos da Microsoft, a arquitetura foi propositadamente criada para não tornar a empresa refém da Azure (*vendor lock-in*). Como o "motor" do sistema (as APIs e *Workers*) está empacotado em contentores universais (*Azure Container Apps*), todo o código pode ser facilmente migrado para a AWS ou Google Cloud (GCP) a qualquer momento. Da mesma forma, os recursos de apoio escolhidos — como bancos de dados SQL, *caches* em memória e barramentos de mensagens — possuem equivalentes diretos em qualquer outro provedor, garantindo total liberdade de infraestrutura e protegendo o investimento da empresa a longo prazo.

No modelo C4, o entendimento da Visão Target apresenta-se nos seguintes níveis:

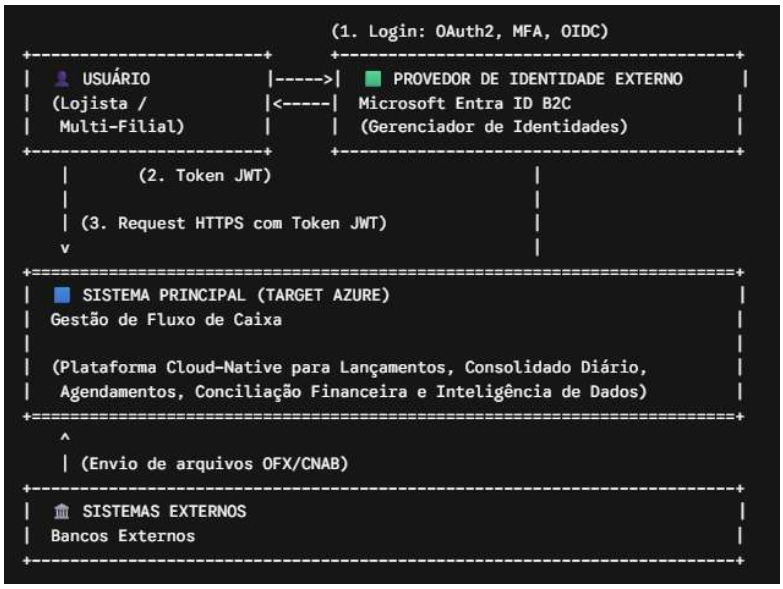


Figura 10– Diagrama de Contexto

A figura 10 representa o nível 1, ilustrando a interação do utilizador e sistemas externos com a nova plataforma hospedada na nuvem da Microsoft Azure.

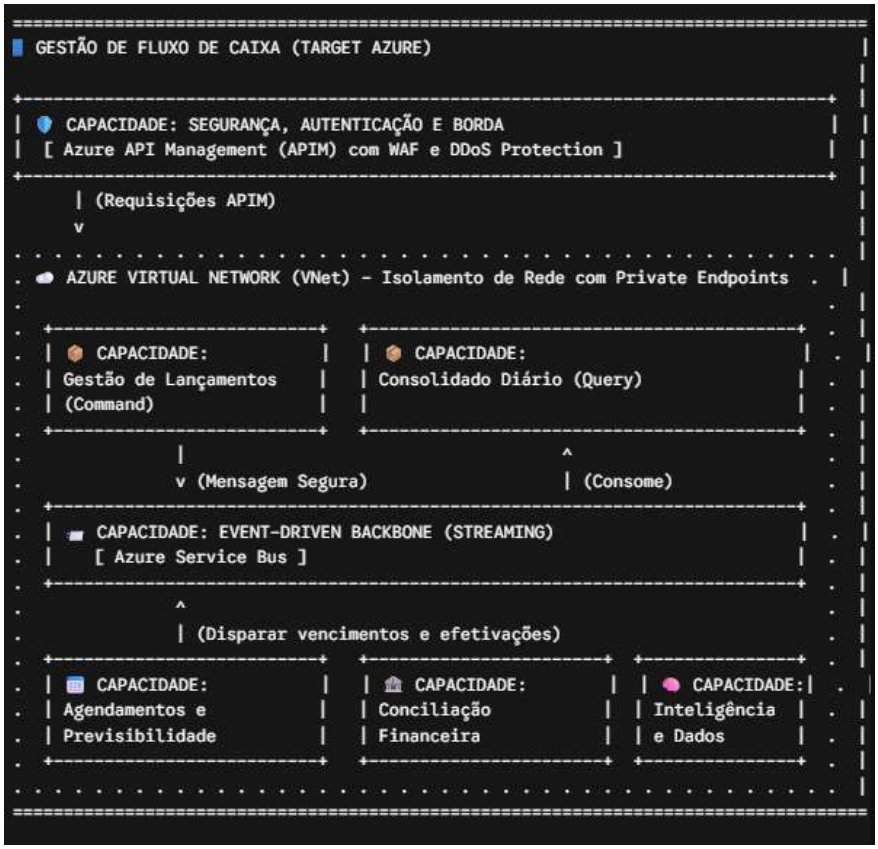


Figura 11– Diagrama de Container

A figura 11 apresenta o nível 2(diagrama de container), demonstrando como o sistema foi decomposto em diferentes capacidades(agrupamentos de serviços) e destaca o perímetro de segurança da rede virtual.

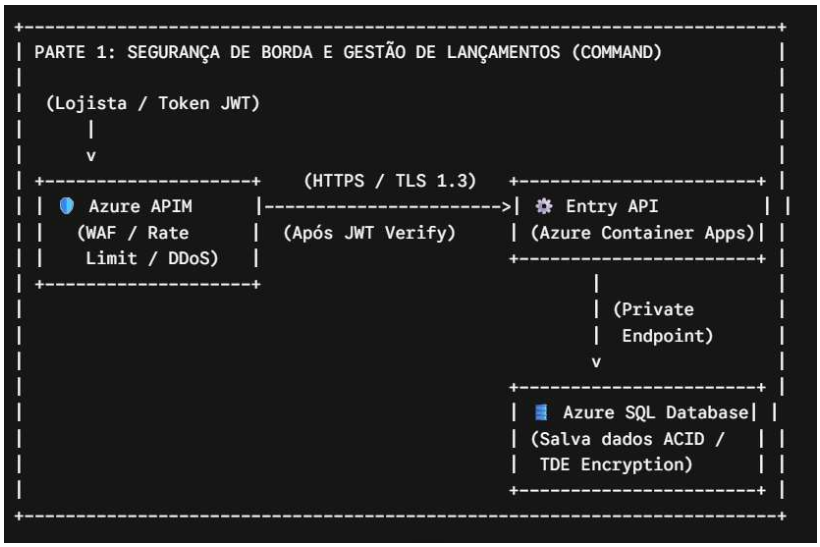


Figura 12– Diagrama de Componentes – Parte 1: Fluxo de Entrada

A figura 12 apresenta o nível 3(diagrama de componentes), destacando o fluxo de entrada: na recepção, validação e persistência segura do lançamento que chega do caixa do lojista.

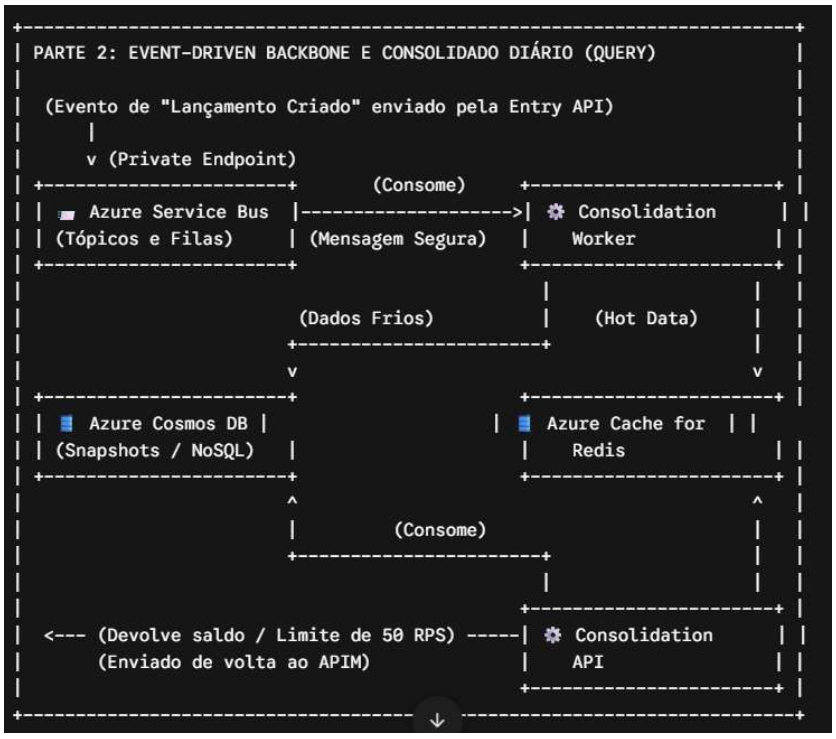


Figura 13– Diagrama de Componentes – Parte 2: Mensageria e Leitura

A figura 13 apresenta o nível 3(diagrama de componentes), cujo foco está no motor assíncrono. Demonstra como o lançamento é transportado pelo barramento de eventos, processado pelo *Worker* e disponibilizado pela consulta rápida.

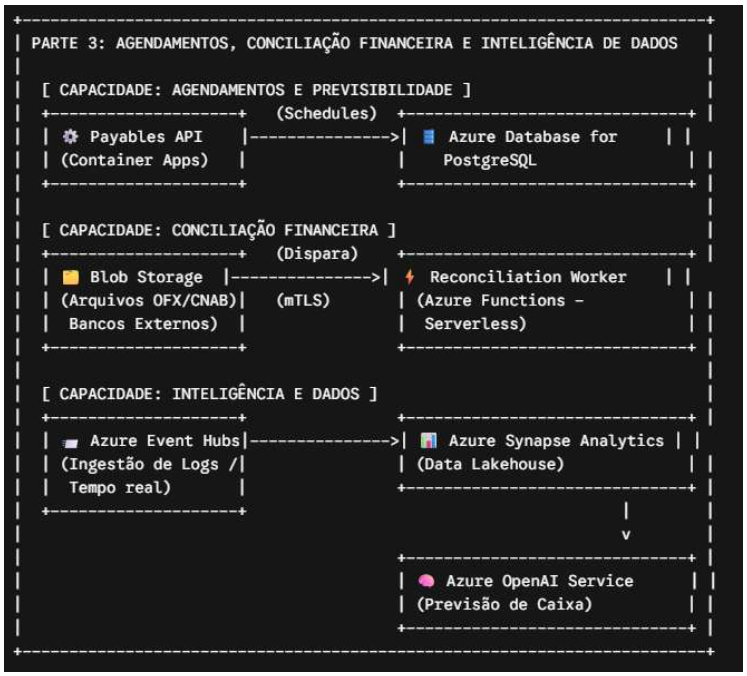


Figura 14– Diagrama de Componentes – Parte 3: Operações de Retaguarda e Dados Avançados

A figura 14 apresenta o nível 3(diagrama de componentes), agrupando as capacidades secundárias, porém essenciais ao negócio, que são executadas em paralelo, como integrações bancárias, agendamentos e inteligência.

A partir do fluxo de execução da arquitetura na visão Target, os requisitos funcionais, não-funcionais e técnicos do sistema são:

Requisitos funcionais:

- **RF-01 (Registro de Movimentação):** O sistema deve fornecer uma API (Entry API) capaz de receber, validar e registar lançamentos financeiros (débitos e créditos) iniciados pelo lojista.
- **RF-02 (Ingestão Legada - CDC):** O sistema deve capturar de forma assíncrona, através de *Change Data Capture* (CDC), as alterações realizadas na base de dados legada e transformá-las em eventos de negócio.
- **RF-03 (Consolidação de Saldo):** O sistema deve possuir um processo em *background* (*Worker*) que consuma os eventos de lançamentos e processe matematicamente a consolidação do saldo financeiro diário por estabelecimento.
- **RF-04 (Consulta de Relatório):** O sistema deve fornecer uma API (Consolidation API) para que o lojista consulte o saldo consolidado atualizado, de forma independente do fluxo de escrita.
- **RF-05 (Autenticação Federada):** O sistema deve autenticar os utilizadores (lojistas e multifiliais) utilizando um provedor de identidade centralizado (Microsoft Entra ID B2C) gerando *tokens* JWT com base em *Claims* e RBAC (Role-Based Access Control).
- **RF-06 (Agendamento de Pagáveis):** O sistema deve permitir o agendamento de contas a pagar através de uma API dedicada (Payables API) e base de dados relacional própria.

- **RF-07 (Conciliação Bancária Automatizada):** O sistema deve processar arquivos padrão bancário recebidos em *storage*, disparando automaticamente uma rotina de conciliação para cruzar o caixa com a conta bancária.

Requisitos não-funcionais:

- **RNF-01 (Desempenho e Throughput):** As APIs devem suportar picos de tráfego de até 50 Requisições Por Segundo (RPS) sem degradação de tempo de resposta. As requisições de consulta de saldo devem ter complexidade temporal O(1) (Tempo constante).
- **RNF-02 (Resiliência e Tolerância a Falhas):** A perda máxima tolerável de eventos/transações é fixada no rigoroso limite de 5%. O sistema de mensageria deve atuar como *buffer* (retenção em disco/fila) para garantir que picos de carga não resultem em perda de dados caso as APIs de leitura estejam indisponíveis.
- **RNF-03 (Isolamento de Domínio / CQRS):** A disponibilidade do domínio de escrita (Gestão de Lançamentos) não pode ser impactada pela indisponibilidade ou lentidão do domínio de leitura (Consolidado Diário), eliminando o "Acoplamento Síncrono Total" do legado.
- **RNF-04 (Segurança Zero-Trust):** A arquitetura deve implementar isolamento total de rede pública. Todo o tráfego interno entre microsserviços e bases de dados deve ocorrer em uma Virtual Network (VNet) através de *Private Endpoints*.
- **RNF-05 (Proteção e Criptografia):** A borda deve ser protegida por Web Application Firewall (WAF) mitigando ataques L7. Os dados em trânsito devem usar mTLS/TLS 1.3 e dados em repouso exigem criptografia TDE (*Transparent Data Encryption).* Acesso a bancos de dados deve usar identidades gerenciadas (sem *passwords* em código).
- **RNF-06 (Elasticidade):** O processamento de retaguarda (APIs e Workers) deve escalar horizontalmente de forma automática (Auto-scaling / *Scale-to-Zero*) com base no volume de eventos da fila de mensageria ou carga de CPU.

Requisitos técnicos:

- **RT-01 (Stack de Desenvolvimento):** O código fonte das APIs e do Worker deve ser escrito utilizando a linguagem de domínio do time de engenharia(programação).
- **RT-02 (Bancos de Dados Poliglotas):** O armazenamento deve respeitar a especialidade de cada domínio:
    - *Command:* SQL Server (garantia transacional ACID).
    - *Query (Hot Data):* Redis Cache (para consulta ultrarrápida do saldo).
    - *Query (Cold Data):* MongoDB (NoSQL) para armazenamento de histórico de *snapshots*.
- **RT-03 (Mensageria MVP):** O transporte assíncrono de eventos e o padrão *Transactional Outbox* devem ser implementados sobre o **Apache Kafka** (devido à sua alta retenção em disco comparado ao RabbitMQ).

- **RT-04 (Conteinerização):** A execução e orquestração local (ambiente de desenvolvimento) de toda a infraestrutura e código deverá ser feita exclusivamente via Docker/Docker Compose.
- **RT-05 (Cloud Provider Oficial):** Toda a infraestrutura Target será migrada para serviços geridos (PaaS) e Serverless no ecossistema Microsoft Azure ou para o provedor em nuvem de decisão da empresa , com a restrição de manter o código de aplicação agnóstico (desacoplado do *vendor*) via uso de *Containers*.
- **RT-06 (Mapeamento de Serviços PaaS):**
    - *API Gateway & Rate Limiting:* Azure API Management (APIM).
    - *Computação (APIs e Workers):* Azure Container Apps (para o CQRS central) e Azure Functions (para a Conciliação via *Trigger)*.
    - *Bancos de Dados Gerenciados:* Azure SQL Database, Azure Cosmos DB e Azure Cache for Redis.
    - *Mensageria PaaS:* Azure Service Bus (substituindo o Kafka) e Azure Event Hubs para ingestão maciça de logs.
- **RT-07 (Observabilidade Unificada):** É estritamente proibido o armazenamento de logs textuais em disco local. Toda a aplicação deve ser instrumentada com **OpenTelemetry**, enviando Traces, Logs e Metrics para o Grafana (na Visão Tática) e migrando para o Azure Log Analytics/Application Insights na Visão Target.
- **RT-08 (Gestão de Segredos):** As credenciais, chaves de API e certificados da arquitetura Target deverão ser obrigatoriamente injetados no tempo de execução através do Azure Key Vault.

Na proposta de apresentar os ganhos por capacidades de negócio e componentes utilizados na Visão Target, segue os tópicos agrupados por:

1. **Capacidade: Segurança, Autenticação e Borda**
    - **Componentes:** Entra ID B2C, WAF, API Management (APIM), Key Vault, Virtual Network (VNet).
    - **Entrega de Valor:** O **Entra ID** gerencia identidades e MFA (autenticação forte). O **WAF** protege contra-ataques L7. O **APIM** atua como fachada segura, aplicando a regra estrita de **limite de 50 RPS (Rate Limiting)** exigida no negócio. A **VNet** com *Private Endpoints* garante isolamento de rede: as APIs não são expostas à internet pública.
2. **Capacidade: Gestão de Lançamentos (Command)**
    - **Componentes:** Entry API (Azure Container Apps), Azure SQL Database (TDE Encryption).
    - **Entrega de Valor:** O *Container Apps* permite escalabilidade elástica baseada em eventos (KEDA). O Azure SQL gerenciado fornece criptografia em repouso (TDE) nativa, essencial para dados financeiros.
3. **Capacidade: Event-Driven Backbone (Streaming)**

- **Componentes:** Azure Service Bus.
- **Entrega de Valor:** Substitui o Kafka por um serviço totalmente gerenciado. Garante entrega segura de mensagens (Topics e Filas) com alta disponibilidade nativa.

4. **Capacidade: Consolidado Diário (Query)**
   - **Componentes:** Consolidation Worker e API (Container Apps), Azure Cosmos DB, Azure Cache for Redis.
   - **Entrega de Valor:** O **Cosmos DB** entrega latência de milissegundos e distribuição global. O **Redis** gerenciado atende requisições instantâneas. Esta tríade garante a entrega do relatório financeiro independentemente da volumetria de entrada.
5. **Novas Capacidades Agregadas (Evolução do Produto):**
   - **Agendamentos e Previsibilidade:** Payables API e Azure Database for PostgreSQL para gestão de contas a pagar.
   - **Conciliação Financeira:** *Azure Blob Storage* recebe arquivos OFX/CNAB de bancos, disparando eventos para um *Reconciliation Worker (Azure Functions - Serverless)* integrar o caixa físico com a conta bancária.
   - **Inteligência e Dados:** *Azure Event Hubs* faz ingestão em tempo real para o *Synapse Analytics (Data Lakehouse)*, alimentando o **Azure OpenAI Service** para gerar previsões financeiras ao lojista com base em IA, respeitando a privacidade dos dados.

A tabela a seguir apresenta a visão de negócio e capacidade técnica com foco em valor de entrega:

Domínio de Negócio	Capacidade Técnica Azure	Valor Entregue ao Usuário
Operações de Caixa	Command (Container Apps + SQL)	Lançamento rápido e seguro de transações.
Relatórios Gerenciais	Query (CosmosDB + Redis)	Saldo em tempo real, sem lentidão na loja.
Automação Financeira	Conciliação (Blob + Functions)	Fim do trabalho manual com planilhas de banco.
Inovação de Negócio	Inteligência (Synapse + OpenAI)	Previsão de quebra de caixa e dicas financeiras.

Na proposta de mensurar os custos mensais da Visão Target, a estimativa mensal está baseada em *consumo Pay-As-You-Go:*

- **Azure API Management (Standard tier):** média de R$3.500,00/mês (Essencial para WAF e Rate Limit).
- **Azure Container Apps (Entry, Worker, Consolidation):** Baseado em consumo. Estimativa: média entre R$ 750,00 a R$ 1.250/mês.

- **Azure SQL Database (Serverless ou Basic vCore):** média entre R$500,00 a R$1.000,00/mês.
- **Azure Service Bus (Standard):** média de R$50,00/mês.
- **Azure Cosmos DB (Serverless / RU provisionada baixa):** média entre R$250,00 – a R$2.500,00/mês.
- **Azure Cache for Redis (Basic):** média de R$200,00/mês.
- **Azure Functions / Blob / Key Vault / Entra ID B2C:** Custos residuais baseados em pequenos volumes de transação (Ficam na faixa gratuita ou < R$250,00/mês).
- **Data & AI (Synapse + OpenAI):** média entre R$1.000,00 a R$2.000,00/mês (Depende fortemente da volumetria de queries do LLM).
- **Total Estimado Target:** média entre R$6.500,00 a R$10.500,00/mês. (O valor nominal é superior à infra IaaS, mas elimina custos operacionais de equipe de infraestrutura gerenciando Kafka, clusters Mongo, licenças SQL Server Enterprise, patching de SO e gerenciamento de rede, entregando um ROI positivo).

Seção 3: Justificativa

A decisão de dividir a modernização do sistema em duas etapas (Visão Tática e Visão Target) fundamenta-se na necessidade de estancar as dores urgentes do negócio sem incorrer nos altíssimos riscos e na demora de uma migração total. A adoção da Visão Tática com os padrões CQRS e arquitetura orientada a eventos (Event-Driven) resolve o problema crítico de acoplamento síncrono e travamentos de banco de dados do ambiente legado. Esta abordagem isola a capacidade de Gestão de Lançamentos da de Consolidado Diário, utilizando CDC (Change Data Capture) para a ingestão contínua de dados sem impactar a operação.

Para a arquitetura definitiva, a Visão Target justifica-se pela meta estratégica de atingir a maturidade máxima em resiliência, segurança e escalabilidade, transferindo o fardo operacional da gestão de infraestrutura para serviços gerenciados (PaaS e Serverless). Esta fundação técnica cumpre com rigor os requisitos não-funcionais estabelecidos: suporte a 50 RPS (Requisições Por Segundo), proteção contra perdas e latência baixíssima para relatórios.

Adicionalmente, justifica-se a substituição do armazenamento legado de *logs* em arquivos de texto (que causavam paradas por disco cheio) pela adoção do OpenTelemetry. Este padrão de observabilidade permitirá a instrumentação unificada da aplicação, gerando *logs, traces* distribuídos e métricas precisas. Na Visão Tática, esses dados serão consolidados e visualizados no Grafana(e posteriormente no Azure Log Analytics/Application Insights na Visão Target), garantindo visibilidade em tempo real sobre a saúde do sistema e reduzindo o tempo de resposta a incidentes.

Alternativas Rejeitadas: A migração direta e imediata de toda a infraestrutura legada para a nuvem foi rejeitada por apresentar risco extremo de disrupção na frente de caixa e por atrasar a entrega de valor ao comerciante.

Restrições, Premissas e Experimentos:

- **Agnosticismo Tecnológico:** Como a escolha do provedor é uma decisão de gestão, estipulou-se a premissa de empacotar o "motor" da aplicação em contêineres universais. Isso evita o *vendor lock-in*, garantindo que o código possa ser migrado para AWS ou GCP no futuro.

- **Premissa Financeira (FinOps):** Reconhece-se que o modelo de nuvem (Visão Target) altera a estrutura de custos de capital fixo (CapEx) para custos operacionais variáveis (OpEx), com uma estimativa mensal entre R$6.500,00 a R$10.500,00/mês. A aplicação das práticas de **FinOps** é uma premissa obrigatória para justificar este investimento. Através da visibilidade das métricas no Grafana/Log Analytics e da gestão de consumo *Pay-As-You-Go*, será possível otimizar os custos, escalando recursos apenas quando necessário (escala elástica). O custo superior é justificado pelo ROI positivo gerado ao erradicar os custos de manutenção física, licenças de software legado e interrupções nas vendas.

Seção 4: Consequências

A aplicação das decisões arquiteturais propostas resultará no seguinte contexto operacional, introduzindo tanto benefícios claros quanto novos desafios sistêmicos:

Consequências Positivas:

- **Erradicação de Gargalos e Acoplamento:** A separação entre leitura e escrita (CQRS) garante que consultas complexas ou picos de acesso ao relatório gerencial nunca travem o registo de vendas no caixa do lojista.

- **Resiliência e Tolerância a Falhas:** A introdução de mensageria assíncrona criará um sistema onde quedas no serviço de consolidação não resultam em perda de dados de transações (offset de segurança).

- **Observabilidade Proativa:** A equipe de engenharia ganhará rastreabilidade ponta a ponta. Com o OpenTelemetry (por exemplo) gerando as métricas e o Grafana (por exemplo), centralizando a visualização, falhas silenciosas serão detectadas proativamente, e a dor de negócio de "acesso manual a servidores via RDP/SSH para ler arquivos de texto" será eliminada, otimizando o MTTR *(Mean Time to Resolution* – tradução: tempo médio de resolução).

- **Segurança de Nível Corporativo:** A Visão Target eleva a postura de segurança com a introdução do padrão Zero-Trust(confiança zero), utilizando isolamento de rede via Private Endpoints, WAF (Web Application Firewall) e identidades gerenciadas.

- **Controle e Otimização de Custos (FinOps):** A arquitetura habilitará a prática de **FinOps**, permitindo que as equipes de negócios e engenharia colaborem na otimização da infraestrutura. A capacidade de desligar instâncias ociosas (*Scale-to-Zero*) e acompanhar o faturamento por consumo garantirá previsibilidade orçamentária e proteção contra desperdícios (mitigando surpresas na fatura).

Consequências Negativas ou Desafios (Trade-offs):

- **Aumento Exponencial da Complexidade Técnica:** A equipe de desenvolvimento passará a lidar com sistemas distribuídos, eventuais problemas de consistência assíncrona e bancos de dados poliglotas.

- **Aumento da Complexidade de Governança (FinOps):** O modelo *Pay-As-You-Go* exige monitorização constante. Sem uma governança rigorosa de **FinOps**, o custo elástico da nuvem pode facilmente ultrapassar as estimativas mensais em casos de código mal otimizado ou ataques volumétricos não bloqueados pelo *Rate Limiting*.
- **Retrabalho Planejado:** A adoção da Visão Tática de transição exigirá que a equipe crie infraestrutura intermédia (como o Kafka local e instâncias do Grafana) que, posteriormente, serão descartadas e substituídas pelos serviços totalmente gerenciados do Azure ou por outro provedor em nuvem de escolha pela gestão da empresa.

Diante do registro de decisão de arquitetura, no qual apresentou as propostas arquiteturais para o negócio gestão financeira nas visões TÁTICA e TARGET, será contemplado o arquivo **Arquitetura-Solucao.drawio**, cuja proposta é expandir a visualização das visões em questão.